pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: A Trade Blotter

WEEK 3, DAY 5

Created by **Jeff Lenamon**

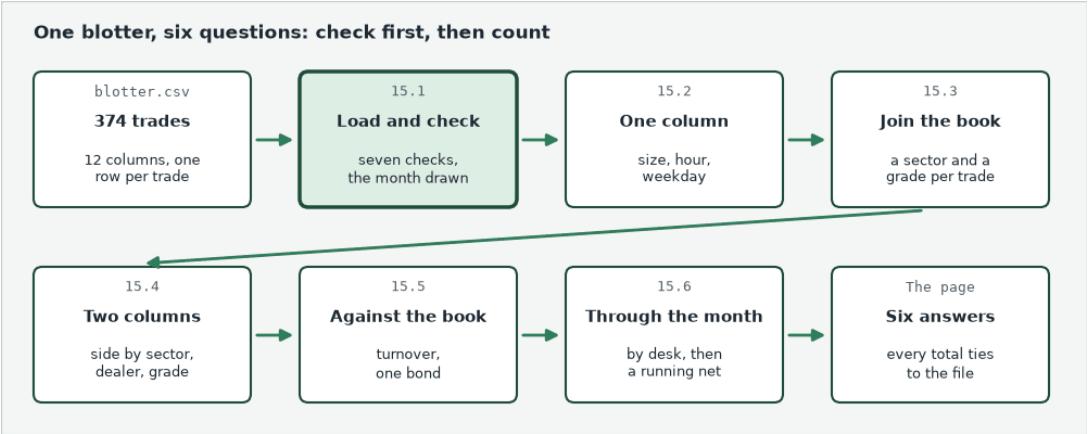
This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note, and a month of trades

Month-end meeting: I need one page on September from the blotter, with every total tied to the file. Describe it: no views on bonds, dealers, or desks.

The trade blotter is the desk's list of every trade it made. Every name in the file is invented.

Check the file before any total.



"trade_blotter.csv" in "data": one row per trade, 2026-09-01 to 2026-09-30, each in a bond of "holdings.csv".

Column	Holds
`trade_id, trader`	the trade; the desk, Desk 1 to Desk 4
`trade_date, trade_time, settle_date`	when it was traded and when it settles
`cusip, ticker, counterparty`	the bond; the dealer on the other side
`side, quantity, price`	BUY or SELL; face; clean price
`principal`	quantity x price / 100, without accrued interest

Face and principal in dollars, prices per 100 of face, totals in millions.

15.1 Seven checks before any total

```
blotter = pd.read_csv(data_folder + 'trade_blotter.csv', parse_dates=['trade_date', 'settle_date'])
print(blotter.shape)
blotter.head(2)
(374, 12)
```

```
ids_unique = blotter['trade_id'].is_unique
blanks = blotter.isna().sum().sum()
repeats = blotter.duplicated().sum()
print('ids unique:', ids_unique, '| blank cells:', blanks, '| repeated rows:', repeats)
print(blotter['side'].value_counts().to_dict())
assert ids_unique and blanks == 0 and repeats == 0, 'a repeated id, a blank, or a repeated row'
assert blotter['side'].isin(['BUY', 'SELL']).all(), 'side holds something other than BUY or SELL'
ids unique: True | blank cells: 0 | repeated rows: 0
{'BUY': 197, 'SELL': 177}
```

Every id once, no blank, no repeat, and no stray "Buy": 197 buys and 177 sells. Print for the reader, assert for the run nobody watches.

"T0001" bought 500,000 of face at 103.053: 500,000 x 103.053 / 100 is its 515,265 of principal.

```
largest_gap = (blotter['quantity'] * blotter['price'] / 100 - blotter['principal']).abs().max()
odd_lots = (blotter['quantity'] % 5000 != 0).sum()
holdings = pd.read_csv(data_folder + 'holdings.csv')
in_book = blotter['cusip'].isin(holdings['cusip'])
print('largest gap, dollars:', round(largest_gap, 2), '| odd lots:', odd_lots,
      '| in the book:', in_book.sum())
print('bonds traded:', blotter['cusip'].nunique(), 'of', len(holdings),
      '| trade dates:', blotter['trade_date'].nunique())
assert largest_gap < 0.005 and odd_lots == 0 and in_book.all(), 'a tie-out failed'
largest gap, dollars: 0.0 | odd lots: 0 | in the book: 374
bonds traded: 128 of 200 | trade dates: 21
```

Every principal ties to the cent, every quantity is a round lot, and every trade is in a bond the book holds: 128 of the 200 traded, on 21 dates.

Excel side by side. The blotter in A to L, rows 2 to 375; "holdings" on a second sheet.

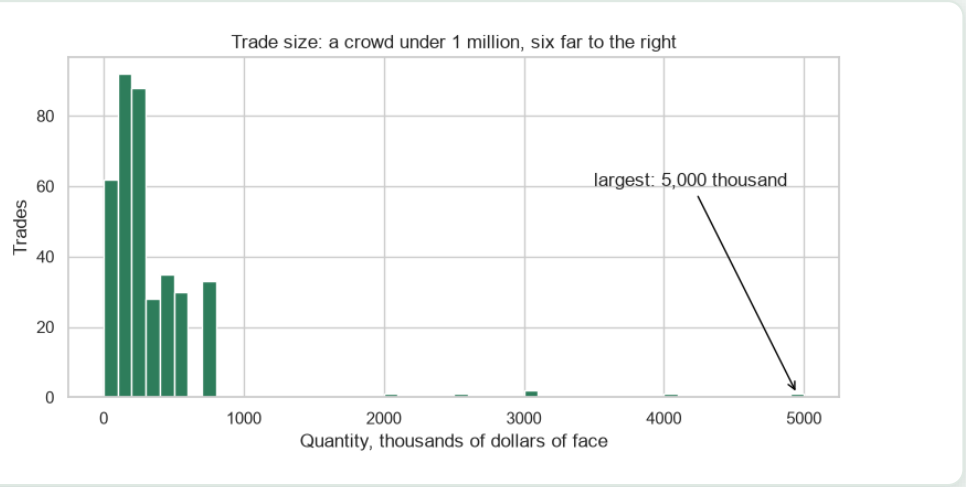
Check	pandas	Excel
Blank cells	`isna().sum().sum()`	=COUNTBLANK(A2:L375), 0
Round lots	`quantity % 5000`	=MOD(H2,5000) filled down, 0 on every row
In the book	`isin()`	=COUNTIF(holdings!B:B,E2) filled down, 1 on every row
Stop on a failure	`assert`	none:
		a check cell, =IF(COUNTBLANK(A2:L375)=0,"ok","STOP"), shows ok but stops nothing

A clean result is a result: seven checks, all passed, all written down.

15.2 How big is a trade, and six far from the rest

```
blotter['quantity'].describe().round()
count        374.0
mean      312968.0
std       444799.0
min        10000.0
25%       100000.0
50%       200000.0
75%       400000.0
max      5000000.0
Name: quantity, dtype: float64
```

Median 200,000, mean 312,968: skewed right, as in Visualization. The middle half runs 100,000 to 400,000, and the largest is 5,000,000.




```
q1, q3 = blotter['quantity'].quantile(0.25), blotter['quantity'].quantile(0.75)
upper_limit = q3 + 1.5 * (q3 - q1)
flagged = blotter[blotter['quantity'] > upper_limit]
share = flagged['principal'].sum() / blotter['principal'].sum() * 100
print('limit:', upper_limit, '| flagged:', len(flagged))
print('principal, millions:', round(flagged['principal'].sum() / 1_000_000, 2),
      '|', round(share, 1), 'percent of the dollars')
flagged[['trade_id', 'trade_date', 'ticker', 'side', 'quantity', 'price']]
limit: 850000.0 | flagged: 6
principal, millions: 19.41 | 16.5 percent of the dollars
```

	trade_id	trade_date	ticker	side	quantity	price
50	T0051	2026-09-03	RULL	SELL	4000000	103.059
137	T0138	2026-09-11	THAN	SELL	2000000	94.181
150	T0151	2026-09-14	DRAH	BUY	2500000	94.101
159	T0160	2026-09-15	CLAE	SELL	5000000	106.702
236	T0237	2026-09-21	PLER	BUY	3000000	98.307
366	T0367	2026-09-30	VRAN	SELL	3000000	92.203

Past the 850,000 limit, six trades of 2 to 5 million carry 16.5 percent of the dollars.

```
month_end_prices = holdings[['cusip', 'price']].rename(columns={'price': 'month_end_price'})
gaps = pd.merge(flagged, month_end_prices, on='cusip')
print('largest gap, points:', round((gaps['price'] - gaps['month_end_price']).abs().max(), 3))
largest gap, points: 0.719
```

Each flagged price is within 0.719 points of its month end, and each principal tied: six real trades, kept.

The hour from text, and the week in order

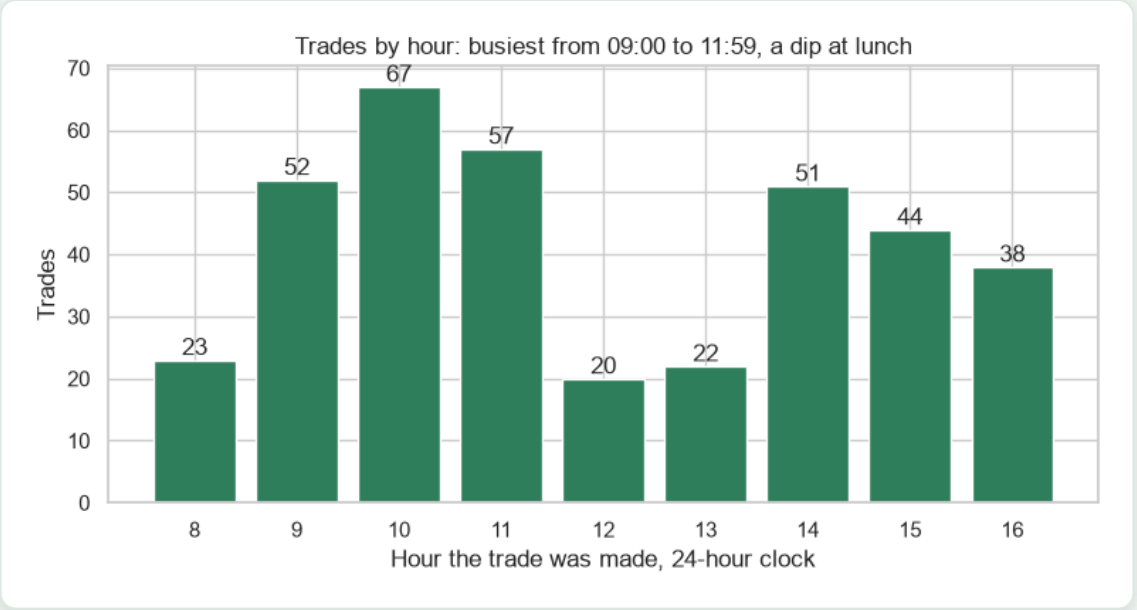
```
blotter['trade_time'].dt.hour
AttributeError: Can only use .dt accessor with datetimelike values
```

Read the last line, as on week 1's Day 3: ".dt" needs dates or times, and "trade_time" is text.

```
as_time = pd.to_datetime(blotter['trade_time'], format='%H:%M')
blotter['hour'] = as_time.dt.hour
print(as_time.iloc[0], '| hour:', blotter['hour'].iloc[0])
print('first:', blotter['trade_time'].min(), '| last:', blotter['trade_time'].max())
1900-01-01 08:04:00 | hour: 8
first: 08:04 | last: 16:59
```

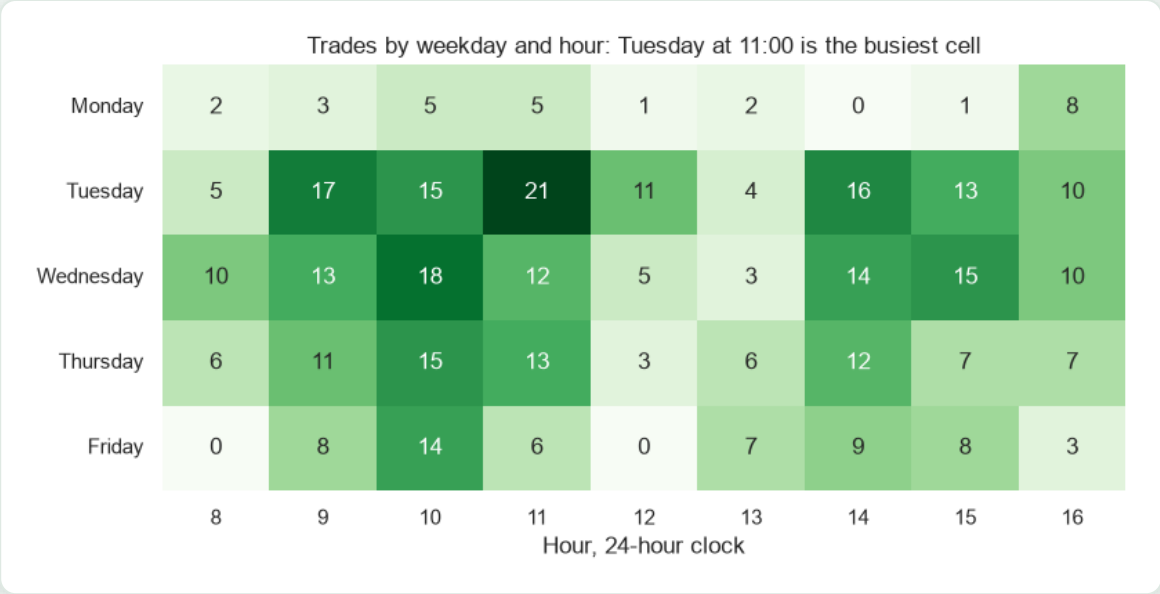
Trading ran from 08:04 to 16:59.

```
week = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']
blotter['weekday'] = pd.Categorical(blotter['trade_date'].dt.day_name(),
                                    categories=week, ordered=True)
blotter['weekday'].head(2)
0    Tuesday
1    Tuesday
Name: weekday, dtype: category
Categories (5, str): ['Monday' < 'Tuesday' < 'Wednesday' < 'Thursday' < 'Friday']
```



10:00 is busiest with 67, between 09:00 with 52 and 11:00 with 57. Lunch is quiet, and 14:00 climbs back to 51.

Weekday by hour



Tuesday at 11:00 holds 21; Friday has none at 08:00 or 12:00.

Excel side by side.

Job	pandas	Excel
The hour	`.dt.hour`	=HOUR(C2), 8:
		Microsoft 365 opens 08:04 as a time, not text
The weekday	`.dt.day_name()`	=TEXT(B2,"dddd"), Tuesday
Weekday by hour	`pd.crosstab()`	a PivotTable:
		weekday in Rows, hour in Columns, Count of trade_id in Values
Size histogram	`ax.hist()`	Insert, Statistic Chart, Histogram (Excel 2016 and later)

Mornings are busiest. The note's other questions need a sector, and the blotter has none.

15.3, 15.4 A sector for every trade, then net by sector

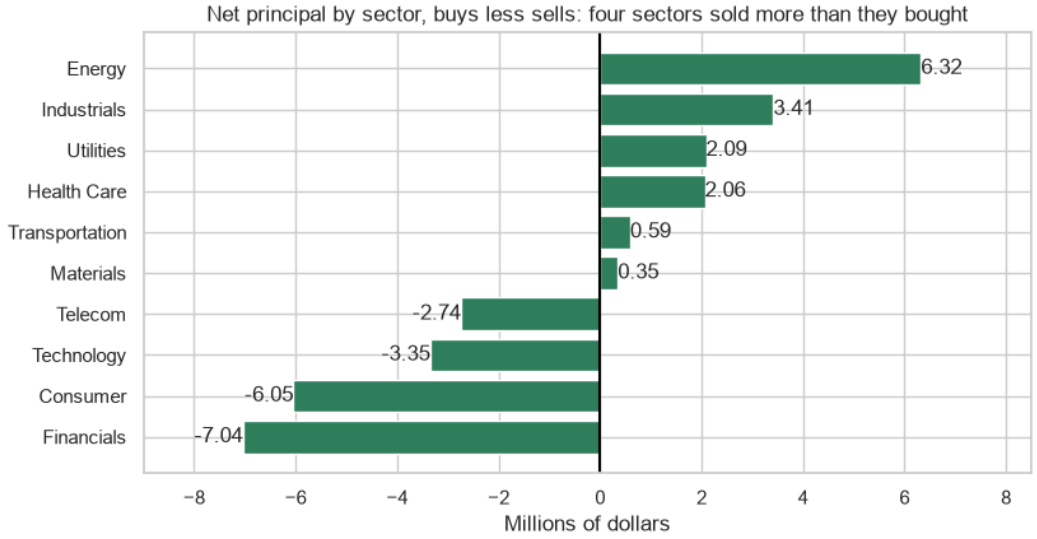
```
bond_details = holdings[['cusip', 'issuer', 'sector', 'rating', 'price', 'market_value']]
bond_details = bond_details.rename(columns={'price': 'month_end_price'})
trades = pd.merge(blotter, bond_details, on='cusip', how='left', validate='many_to_one')
ratings = pd.read_csv(data_folder + 'ratings.csv')
trades = pd.merge(trades, ratings[['rating', 'grade']], on='rating', how='left',
                  validate='many_to_one')
print(len(blotter), 'in |', len(trades), 'out |', trades.shape[1], 'columns')
print('blank sector or grade:', trades[['sector', 'grade']].isna().sum().sum())
assert len(trades) == len(blotter), 'the join added or lost a trade'
374 in | 374 out | 20 columns
blank sector or grade: 0
```

The count held: "cusip" is unique in the holdings, and every trade found its bond in 15.1.

374 in, 374 out: one row is still one trade.

```
by_sector = pd.pivot_table(trades, values='principal', index='sector', columns='side',
                           aggfunc='sum') / 1_000_000
by_sector['net'] = by_sector['BUY'] - by_sector['SELL']
by_sector['total'] = by_sector['BUY'] + by_sector['SELL']
by_sector = by_sector.sort_values('net')
```

Consumer traded the most, 20.13 million, for a net of -6.05; Materials' net of 0.35 hides 12.18 traded.

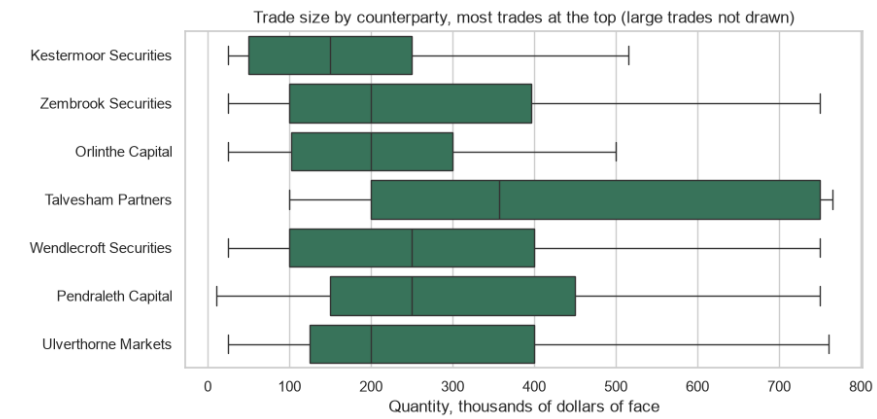


Financials sold the most net, -7.04; Energy bought the most, 6.32.

Counterparties by count and by dollars

counterparty	trade_count	principal	median_size
Kestermoor Securities	86	15.95	150000.0
Zembrook Securities	66	18.58	200000.0
Orlinthe Capital	63	17.62	200000.0
Talvesham Partners	58	36.33	357500.0
Wendlecroft Securities	47	12.18	250000.0
Pendraleth Capital	27	8.61	250000.0
Ulverthorne Markets	27	8.20	200000.0

Kestermoor Securities leads by count, 86 trades for 15.95 million; Talvesham Partners by dollars, 36.33 million in 58.



Kestermoor's median line sits at 150 thousand, Talvesham's at 357.5: fewer, larger trades carry more dollars.

Excel side by side.

Job	pandas	Excel
Sector of a trade	`pd.merge(how='left')`	=XLOOKUP(E2,holdings!B:B,holdings!E:E) in O, Excel 2021 and Microsoft 365 (INDEX and MATCH before);
		it takes the first match silently
Net by sector	`pivot_table(), BUY less SELL`	=SUMIFS(J:J,O:O,"Financials",G:G,"BUY") less the same with "SELL", -7,042,969.65
By counterparty	`named aggregation`	a PivotTable with Count and Sum;
		it has no Median, so =MEDIAN(IF(K2:K375="Kestermoor Securities",H2:H375)), 150,000 (Ctrl+Shift+Enter before Excel 2021)
Size by group	`sns.boxplot()`	Insert, Statistic Chart, Box and Whisker (Excel 2016 and later)

Say whether a figure is a count or dollars, a net or a total.

15.5 Turnover: traded over held

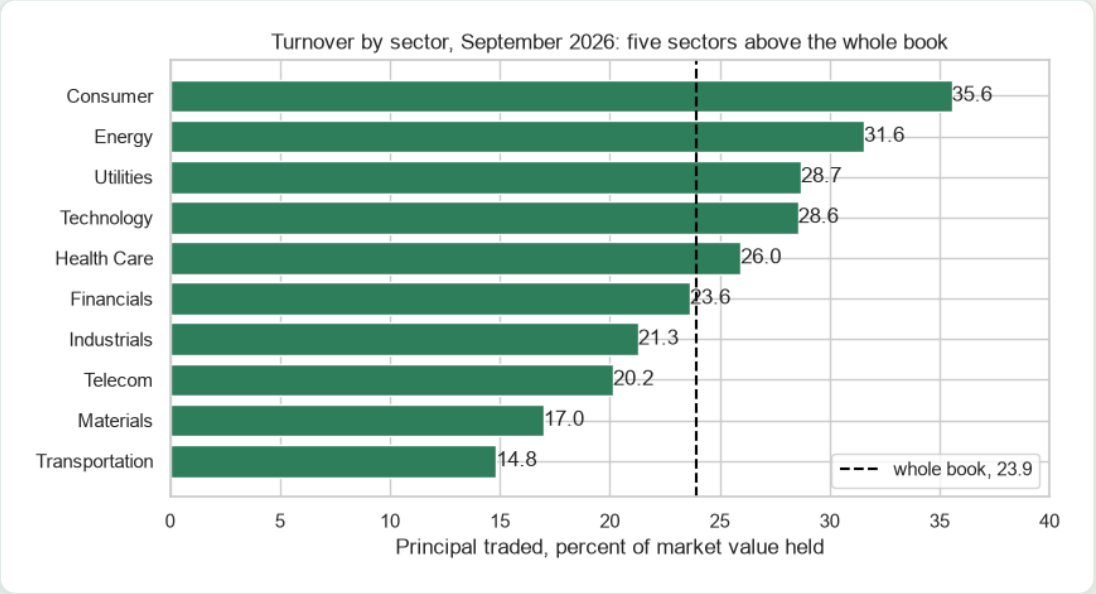
Turnover here is principal traded in the month, buys plus sells, as a percent of market value held at month end. Funds publish a different rate: the smaller of a year's purchases and sales over the monthly average value of the securities held (SEC Form N-1A, Item 13, Instruction 4(d)), so the two do not compare.

Source: SEC Form N-1A, https://www.sec.gov/about/forms/formn-1a.pdf

```
traded = trades.groupby('sector')['principal'].sum() / 1_000_000
held = holdings.groupby('sector')['market_value'].sum() / 1_000_000
turnover = pd.DataFrame({'traded': traded, 'held': held})
turnover['turnover_pct'] = turnover['traded'] / turnover['held'] * 100
whole_pct = traded.sum() / held.sum() * 100
print('whole book:', round(whole_pct, 1), 'percent')
turnover.sort_values('turnover_pct', ascending=False).round(2)
whole book: 23.9 percent
```

sector	traded	held	turnover_pct
Consumer	20.13	56.63	35.55
Energy	12.26	38.83	31.56
Utilities	8.32	29.01	28.70
Technology	8.66	30.31	28.56
Health Care	10.10	38.91	25.96
Financials	16.13	68.22	23.65
Industrials	11.18	52.54	21.29
Telecom	11.26	55.85	20.15
Materials	12.18	71.70	16.99
Transportation	7.24	48.82	14.84

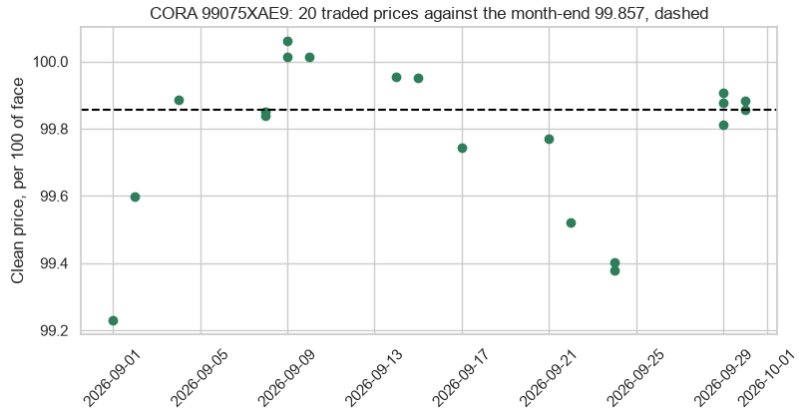
Consumer turned over the most, 35.55 percent; Transportation the least, 14.84; Materials, the largest holding, 16.99.



The most traded bond, against its month end

```
trades_per_bond = trades['cusip'].value_counts()
print(trades_per_bond.head(3).to_dict())
print('traded once:', (trades_per_bond == 1).sum(),
      '| five times or more:', (trades_per_bond >= 5).sum())
top_bond = trades[trades['cusip'] == trades_per_bond.index[0]]
top_bond[['ticker', 'issuer', 'sector', 'rating']].iloc[0]
{'99075XAE9': 20, '99114KAB6': 16, '99007HAA5': 14}
traded once: 57 | five times or more: 25
ticker               CORA
issuer    Corvundra Brands
sector           Consumer
rating                 BB
```

```
simple = top_bond['price'].mean()
weighted = (top_bond['price'] * top_bond['quantity']).sum() / top_bond['quantity'].sum()
month_end = top_bond['month_end_price'].iloc[0]
print('face:', top_bond['quantity'].sum(),
      '| prices:', top_bond['price'].min(), 'to', top_bond['price'].max())
print('simple:', round(simple, 3), '| weighted:', round(weighted, 3), '| month end:', month_end)
face: 3860000 | prices: 99.229 to 100.06
simple: 99.777 | weighted: 99.742 | month end: 99.857
```



"99075XAE9", Corvundra Brands (CORA), a BB Consumer bond, traded 20 times; 57 bonds traded once.

The weighted average, 99.742, sits under the simple 99.777: more face traded at the lower prices. Both sit under the month end, 99.857: say which a page quotes.

Excel side by side.

Job	pandas	Excel
Turnover	`traded / held * 100`	=SUMIF(O2:O375,"Consumer",J2:J375)/SUMIF(holdings!E2:E201,"Consumer",holdings!S2:S201)*100, 35.55
Weighted price	`(price * quantity).sum() / quantity.sum()`	=SUMPRODUCT((E2:E375="99075XAE9")*I2:I375*H2:H375)/SUMIF(E2:E375,"99075XAE9",H2:H375), 99.742

23.9 percent of the book changed hands. Last, the month in order.

15.6 Net by desk, then a running net

```
by_desk = pd.pivot_table(trades, values='principal', index='trader', columns='side',
                         aggfunc='sum') / 1_000_000
by_desk['net'] = by_desk['BUY'] - by_desk['SELL']
```

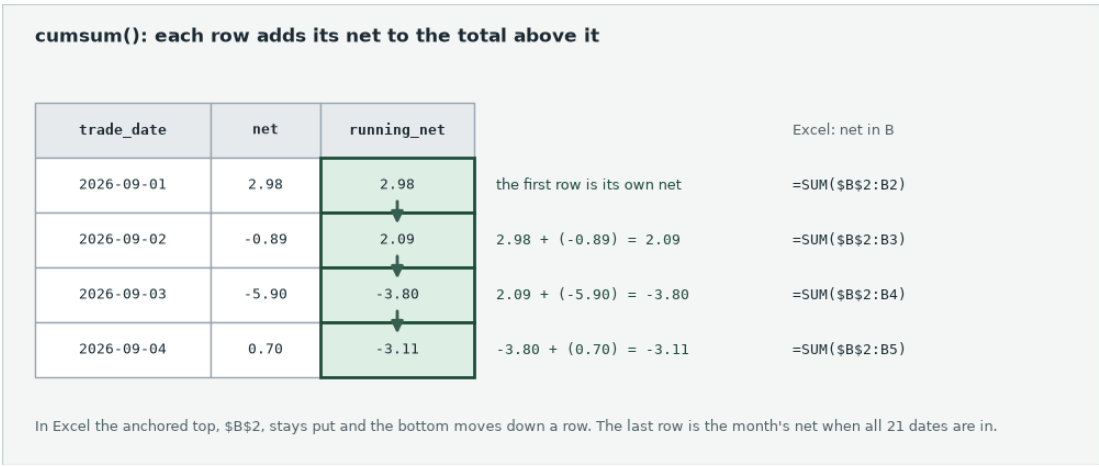
trader	BUY	SELL	net
Desk 1	9.75	14.71	-4.96
Desk 2	22.86	12.77	10.09
Desk 3	17.04	20.44	-3.40
Desk 4	6.91	13.01	-6.10

Desk 2 net bought 10.09 million; Desks 1, 3, and 4 net sold.

Excel side by side.

Job	pandas	Excel
Signed principal	`BUY less SELL`	=IF(G2="BUY",J2,-J2) in Q, filled down

trade_date	BUY	SELL	net	running_net
2026-09-01	5.04	2.06	2.98	2.98
2026-09-02	1.88	2.77	-0.89	2.09
2026-09-03	1.27	7.17	-5.90	-3.80
2026-09-04	2.30	1.60	0.70	-3.11



A running total adds each row to every row above it: "cumsum()", NumPy Part 1's "np.cumsum" on a column.

```
daily = pd.pivot_table(trades, values='principal', index='trade_date', columns='side',
                       aggfunc='sum', fill_value=0) / 1_000_000
daily['net'] = daily['BUY'] - daily['SELL']
daily['running_net'] = daily['net'].cumsum()
```

Predict first: decide, then run each in the notebook

Predict 1

Each trade settles one business day after its trade date. How many different values does "(blotter['settle_date'] - blotter['trade_date']).dt.days" hold? A) one: 1 B) two: 1, and 3 for a Friday C) three

```
gap_days = (blotter['settle_date'] - blotter['trade_date']).dt.days
print(gap_days.value_counts().to_dict())
```

Predict 2

Wednesdays had 100 trades in September, Thursdays 80. Which had more trades per trading day? A) Wednesday B) Thursday C) the same

```
by_weekday = blotter.groupby('weekday', observed=True)['trade_date'].agg(['count', 'nunique'])
by_weekday['per_day'] = by_weekday['count'] / by_weekday['nunique']
by_weekday
```

Predict 3

"pd.merge(holdings, blotter, on='cusip', how='left')" puts the 200 bonds on the left. How many rows come back? Write a number.

```
other_way = pd.merge(holdings, blotter, on='cusip', how='left')
print(len(other_way), 'rows |', other_way['trade_id'].isna().sum(), 'with no trade')
```

Predict first: net, market value, and the lowest point

Predict 4

The desk made 197 buys and 177 sells. Do the counts say whether it was a net buyer or a net seller in dollars? A) Yes: a net buyer, with 20 more buys B) Yes: a net seller C) No

```
by_side = trades.groupby('side')['principal'].agg(['count', 'sum'])
bought, sold = by_side.loc['BUY', 'sum'], by_side.loc['SELL', 'sum']
print('bought:', round(bought / 1_000_000, 2), '| sold:', round(sold / 1_000_000, 2), 'million')
print('net:', round((bought - sold) / 1_000_000, 2),
      '| total:', round((bought + sold) / 1_000_000, 2), 'million')
```

Predict 5

"trades" carries each bond's "market_value", and the book holds 490.82 million. What does "trades['market_value'].sum()" give? A) 490.82 million B) less: only 128 of the 200 bonds traded C) more

```
print('trades:', round(trades['market_value'].sum() / 1_000_000, 2), 'million')
print('holdings:', round(holdings['market_value'].sum() / 1_000_000, 2), 'million')
```

Predict 6

The month ended at -4.36 million. Where was the running net at its lowest? A) -4.36, at month end B) between 0 and -4.36 C) below -4.36

```
low_day, low = daily['running_net'].idxmin(), daily['running_net'].min()
```

The note, answered

The note asked	The blotter says
How much, how big	374 trades, 117.47 million; median 200,000 of face
Who	Kestermoor most often, 86 trades; Talvesham most dollars, 36.33 million
Changed hands	23.9 percent of the book; Consumer 35.55
When	busiest 09:00 to 11:59; Tuesday 22.40 trades a day

The checks behind every answer:

Check	Result
File	374 trades, 12 columns, 2026-09-01 to 2026-09-30; ids unique, no blank, no repeat
Tie-outs	principal to the cent, round lots, every bond in the book, settle one business day on
Join	374 in, 374 out; nothing changed, hour and weekday added

Week 4 opens with Case Study: Consumer Credit, the same routine on a new kind of credit data.